arXiv:submit/7322934 [cs.AI] 3 Mar 2026

Institutional Memory as Organizational Knowledge: AI Agents That Learn Their Jobs from Experience, Not Instructions

Dhillon Andrew Kannabhiran
dhillon@levelupctf.com

Abstract

We demonstrate that AI agents given 3-line role descriptions and access to consensus-validated institutional memory can autonomously create, harden, calibrate, solve, and learn from cybersecurity challenges—without any domain expertise in their prompts. Using 11 specialized agents organized into 5 departments within a governed organization (CipherForge Labs), we present the first fully autonomous, consensus-governed AI security research loop:

1. **A designer agent** (3-line prompt, zero cryptographic knowledge) generates a functional AES-CBC Padding Oracle challenge.

2. **A hardener agent** (3-line prompt) applies 6 defense layers—20-bit Proof of Work, timing side-channels, JSON casing side-channels, single-use tokens—escalating difficulty from 0.80 to 1.75 across 2 iterations.

3. **A calibrator agent** (3-line prompt) correctly assesses the hardened challenge at difficulty 1.80 (gap = 0.20 from target 2.0).

4. **A quality scorer** (3-line prompt) rates the challenge 93.0/100. Total pipeline time: 508 seconds.

5. **An independent solver agent** (blind, no source code access) identifies the casing side-channel vulnerability, writes a C-compiled Proof of Work solver, deploys 32 parallel oracle workers, and captures the flag in 525.2 seconds (16,384 queries).

6. **The findings are submitted** to a 4-node BFT consensus network, validated (score = 0.88), and committed to institutional memory—now queryable by all future agents.

No agent had cryptographic expertise in its prompt. No human intervened at any stage. The entire cycle—creation, defense, assessment, exploitation, and organizational learning—was governed by BFT consensus with department-scoped RBAC access controls.

This result extends our prior finding[1] that an 18-line "onboarding" prompt with curated institutional memory outperformed a 120-line expert prompt. Here we take that principle to its logical extreme: 11 agents, 5 departments, 20+ pipeline routing states, and a closed feedback loop—all driven by minimal prompts and organizational memory.

[1] Dhillon Andrew Kannabhiran. "Consensus-Validated Memory Improves Agent Performance on Complex Tasks". In: *Technical Report* (2026).

1 Introduction

1.1 The Prompt Engineering Bottleneck

Modern multi-agent AI systems rely heavily on prompt engineering to convey domain expertise, behavioral expectations, and workflow rules to individual agents. A typical production pipeline agent might receive a 50–200 line system prompt encoding:

- Domain-specific knowledge (e.g., cryptographic vulnerability types, difficulty assessment heuristics)
- Behavioral rules (output format, error handling, retry logic)
- Organizational context (who to hand off to, what standards to apply)
- Historical lessons (common failure modes, what works and what does not)

This approach has significant limitations:

- **Knowledge staleness:** Prompts encode knowledge at authoring time. Lessons learned during production are not reflected until a human updates the prompt.
- **Inter-agent inconsistency:** When 11 agents each have independent prompts, organizational knowledge (quality standards, difficulty definitions, pipeline protocols) must be duplicated and manually synchronized across all prompts.
- **Maintenance burden:** As the system accumulates operational experience, prompts grow longer, more complex, and harder to maintain. Each new insight requires identifying which agents need the update.
- **No feedback loop:** Agents that discover useful patterns during operation have no mechanism to share them with the organization.

1.2 Institutional Memory as an Alternative

In our prior work[2], we demonstrated that an agent with an 18-line "onboarding" prompt—containing only job scope, scoring rubric, and the instruction "consult your institutional memories"—achieved **perfect calibration** (gap = 0.0) at the highest difficulty level. This result, which no expert-crafted prompt achieved across 20 runs, established the hypothesis:

> **What if agents don't need to be told HOW to do their jobs? What if they only need to be told WHERE they work and given access to organizational memory?**

This paper confirms that hypothesis at organizational scale and closes the loop: agents not only perform their jobs using institutional memory—**they generate new institutional memory through operational experience, validated by consensus, and available to all future agents**.

1.3 The CipherForge Labs Experiment

We construct a fictional company—CipherForge Labs—that creates CTF (Capture The Flag) cybersecurity challenges. The company has:

- **5 departments:** Design, Evaluation, Quality, Red Team, Executive

[2] Kannabhiran, "Consensus-Validated Memory Improves Agent Performance on Complex Tasks", see n. 1.

- **11 specialized agents:** narrative designer, challenge designer, static analyst, build validator, difficulty calibrator, challenge hardener, repair engineer, smoke tester, solution verifier, quality scorer, and CEO
- **A declarative pipeline** with 20+ routing states handling happy paths, failure/repair loops, hardening cycles, and terminal states
- **Institutional memory** via (S)AGE[3]: a 4-node BFT network where agent observations are summarized, validated by weighted consensus, and committed to a queryable knowledge base organized by department and domain

Each agent receives a **3-line prompt:**

```
You are the [Role] at CipherForge Labs ([Department]).
Consult your institutional knowledge before making decisions.
Your work is evaluated on: [single evaluation criterion].
```

All domain expertise, organizational protocols, quality standards, and workflow rules exist exclusively in the institutional knowledge base—not in any agent's prompt.

1.4 Contributions

1. **The first fully autonomous, consensus-governed AI security research loop**—from challenge creation through exploitation to institutional learning, with zero human intervention and full organizational governance.
2. **Empirical proof that 3-line prompts + institutional memory produce production-quality output:** 93.0/100 quality score, 0.20 calibration gap, 6 defense layers applied, all from agents with no domain expertise in their prompts.
3. **Self-improving adversarial cycle:** The solver's pentest report (par time, vulnerability class, defense bypass techniques) is committed to (S)AGE and available to future designers, hardeners, and calibrators—creating an autonomous improvement loop.
4. **Full-organization identity verification:** 11 agents with 3-line prompts correctly understand their roles, departments, colleagues, pipeline position, executive hierarchy, and institutional knowledge access.
5. **Event-driven agent mesh architecture:** A declarative routing-table-driven pipeline that replaces monolithic orchestration with always-on workers, enabling clear separation of concerns and straightforward extension.

1.5 Hypotheses

- **H1 (Organizational awareness):** Agents with minimal prompts and institutional memory correctly identify their role, department, responsibilities, and organizational context. **Strongly supported** (Section 3).
- **H2 (Functional competence):** Agents with minimal prompts and institutional memory can perform their specialized tasks effectively—designing, hardening, calibrating, and scoring cybersecurity challenges at production quality. **Supported by evidence in this experimental setting** (Section 4).
- **H3 (Emergent expertise):** Domain-specific techniques (cryptographic defense mechanisms, side-channel engineering) emerge from the interaction between LLM training data

[3] Dhillon Andrew Kannabhiran. "Agent Memory Infrastructure: Byzantine-Resilient Institutional Memory for Multi-Agent Systems". In: *Technical Report* (2026).

and institutional memory, without explicit prompt instruction. **Supported with caveats** (Section 4.2).

- **H4 (Closed-loop learning):** Operational experience from one pipeline run can be captured, validated by consensus, and made available to future runs—creating a self-improving organizational knowledge base. **Supported by evidence** (Section 4.4).
- **H5 (Prompt minimalism at scale):** 3-line prompts with institutional memory are sufficient for a full 11-agent, 5-department organization to produce, validate, and learn from complex technical artifacts. **Preliminary confirmation in single-domain setting** (Sections 3–4).

2 System Architecture

2.1 Organizational Structure

CipherForge Labs is organized into 5 departments, each with defined responsibilities, domain scopes, and access controls enforced on-chain by (S)AGE's RBAC system:

Department	Agents	Domains	Clearance
Design	narrative, designer	design.*	1 (basic)
Evaluation	calibrator, hardener, validator, exploit	evaluation.*	1–2
Quality	smoke_test, static_analysis, quality, repair, hint	quality.*	1–2
Red Team	solution_verifier	red_team.*	2 (cross-dept)
Executive	ceo	org.*	3 (admin)

Table 1: CipherForge Labs organizational structure. Access controls are enforced at the (S)AGE network level.

Access controls are enforced at the (S)AGE network level:

- Clearance 1 agents can read/write within their own department
- Clearance 2 agents can read across departments
- Clearance 3 (CEO only) has full organizational read/write access

2.2 Agent Mesh Architecture

The pipeline uses an event-driven mesh architecture where agents are always-on workers with async message inboxes:

```
narrative -> designer -> static_analysis -> validator -> calibrator
                                                            |
                                                +------------+
                                                | gap > 0.75 | gap ok
                                                v            v
                                            hardener     smoke_test
                                                |            |
                                                |         solution_verifier
                                                |            |
                                        static_analysis   quality
                                                            |
                                                      COMPLETE / DISCARD
```

Repair loop: If static_analysis or validator fails with a repairable error, the challenge routes to the repair agent (max 2 attempts), then re-enters static_analysis.

Hardening loop: If calibrated difficulty is >0.75 below target, the hardener iteratively modifies the source code (max 2 passes), then re-enters static_analysis.

All routing decisions are encoded in a declarative routing table:

```
ROUTING_TABLE = {
    ("narrative", "success"):          ("designer", "run"),
    ("designer", "success"):           ("static_analysis", "run"),
    ("static_analysis", "pass"):       ("validator", "run"),
    ("validator", "pass"):             ("calibrator", "calibrate_from_source"),
```

```
    ("calibrator", "gap_acceptable"):  ("smoke_test", "run"),
    ("smoke_test", "pass"):            ("solution_verifier", "verify"),
    ("solution_verifier", "verified"): ("quality", "run"),
    ("solution_verifier", "fail"):     ("quality", "run"),  # soft gate
    ("quality", "staged"):             ("_terminal", "COMPLETE"),
    # ... 11 additional routes for failures, repairs, hardening
}
```

Outcome resolution is centralized in a single function (`resolve_outcome`) that inspects pipeline context after each agent executes, eliminating the nested conditional logic typical of monolithic orchestrators.

2.3 Seed Knowledge

Before any agent executes, 8 seed knowledge entries are submitted to the (S)AGE network and validated by BFT consensus:

1. **Company handbook**—mission, quality standards, flag format, Docker requirements
2. **Scoring rubric**—0–100 scale breakdown (solvability 30pts, calibration 25pts, code quality 20pts, narrative 10pts, static analysis 15pts)
3. **Difficulty definitions**—4-tier scale (Easy 0–0.75, Medium 0.75–1.5, Hard 1.5–2.25, Expert 2.25–3.0) with concrete descriptions
4. **Pipeline workflow**—9-stage handoff protocol with department assignments
5. **Department responsibilities**—cross-department handoff expectations
6. **Crypto AES calibration baselines**—category-specific difficulty benchmarks
7. **Effective hardening techniques**—what works and what does not for difficulty escalation
8. **Challenge generation patterns**—library choices, flag isolation patterns, endpoint design

These entries are submitted by an admin agent, validated through weighted consensus, and committed to the knowledge base. Agents then query this knowledge via domain-tagged similarity search during their pipeline execution.

2.4 Solution Verification (Red Team)

The solution verifier is a pipeline stage between smoke testing and quality scoring. It addresses a critical gap: smoke tests verify that the container starts and flag endpoints respond, but do not verify that the challenge is **actually solvable** using the intended exploit chain.

The solution verifier:

1. Launches a fresh Docker container (non-read-only, as solution scripts may write temp files)
2. Attempts to replay the designer's solution_script via `docker exec`
3. If the fast path fails, falls back to LLM-generated solver scripts based on the walkthrough text
4. Captures the flag via regex matching

5. Reports verification level: `solution_verified` (30pts in quality scoring) or `smoke_tested` (20pts)

This is a **soft gate**—verification failure does not discard the challenge, but downgrades its quality score. This design choice reflects the reality that LLM-generated solution scripts are inherently brittle, and a challenge that passes smoke tests but fails automated replay may still be perfectly solvable by a human—or, as we demonstrate in Section 4.3, by a more capable AI solver given sufficient time.

2.5 CEO and Organizational Learning

The CEO agent serves as the organizational monitor and knowledge curator:

- Tracks KPIs: completion rate, average quality, average calibration gap, repair rate, hardening rate
- Queries (S)AGE for historical performance before each pipeline run
- Summarizes and submits pipeline observations after each run (outcomes, lessons learned, failure patterns)
- Submits external validation results (red team assessments, blind solve reports) to (S)AGE for consensus validation
- Can trigger external validation by writing a JSON request file for independent blind-solve agents

3 Organizational Awareness: Identity Verification

3.1 Experimental Setup

Before running the full pipeline, we conducted an identity verification test to assess whether agents with minimal prompts understand their organizational context. Each of the 11 agents was presented with:

- Their 3-line role prompt
- The full seed knowledge (8 entries, ∼6,800 characters)
- 6 identity questions:
  1. What is your job title and what department do you belong to?
  2. What is CipherForge Labs and what does it do?
  3. What are your specific responsibilities in the challenge pipeline?
  4. Who is the CEO and what role does executive leadership play?
  5. Do you have access to an institutional knowledge base? How does it help you?
  6. What happens to the work product after you finish your part?

All agents used Gemini 3 Flash (`gemini-3-flash-preview`) as their LLM.

3.2 Results

11/11 agents responded correctly to all 6 questions. Key findings:

Job awareness: Every agent correctly identifies their title, department, and evaluation criteria. The solution verifier identifies as Red Team, the calibrator as Evaluation, the CEO as Executive—none of this department assignment information appears in the individual agent prompts.

Company understanding: All 11 agents describe CipherForge Labs as a CTF challenge creation company producing Docker-based challenges with `LEVELUP{}` flag format and difficulty calibration. This knowledge comes entirely from the seed entries.

Pipeline position awareness: Each agent correctly identifies:

- Their predecessor in the pipeline (e.g., smoke tester knows it follows calibration)
- Their successor (e.g., solution verifier knows quality scoring follows)
- Alternative paths (e.g., repair engineer knows failed challenges may return to static analysis)

Executive hierarchy: All agents recognize the CEO as the organizational leader who tracks KPIs and reports to the (S)AGE board. No agent was told about the CEO in their prompt—this knowledge is derived entirely from the organizational seed entries.

Knowledge base awareness: Every agent affirms access to institutional knowledge and correctly describes how it helps them (calibration baselines, scoring rubrics, hardening techniques, etc.). Several agents reference specific seed entries by domain (e.g., calibrator references "calibration baselines for AES").

Handoff understanding: Agents demonstrate correct understanding of the full pipeline flow, including departments they do not belong to. The designer knows about evaluation and quality; the smoke tester knows about calibration and hardening.

3.3 Analysis

These results confirm H1 (organizational awareness) and provide strong support for H5 (prompt minimalism). A 3-line prompt that says "You are the Difficulty Calibrator at CipherForge Labs (Evaluation Department)"—combined with organizational seed knowledge—produces an agent that understands:

- What CipherForge Labs is and what it does
- What the Evaluation Department is responsible for
- Who else is in the department (hardener, validator, exploit)
- Where calibration fits in the 9-stage pipeline
- What the scoring rubric expects of calibrated challenges
- What specific difficulty numbers mean (ECB mode = 0.5–0.75, Padding Oracle caps at 1.8, etc.)
- Who the CEO is and what the (S)AGE board does
- That institutional memory should be consulted before making decisions

None of this information is in the agent's prompt. All of it comes from 8 seed knowledge entries totaling ∼2,500 words.

4 The Closed Loop: AI Designs, AI Hardens, AI Solves, AI Learns

4.1 Pipeline Execution

The CipherForge Labs pipeline was initialized with a fresh (S)AGE network (8 seed knowledge entries, zero operational history) and tasked with producing a crypto_aes challenge at target difficulty 2.0. The full execution:

Stage	**Agent**	**Outcome**	**Time**
Narrative	narrative	"The Orbital Echo" (Aerospace)	8s
Generation	designer	AES-CBC Padding Oracle (d=0.80)	71s
Static Analysis (1st)	static_analysis	Pass—no flag leaks	<1s
Docker Validation	validator	Build + health check pass	8s
Calibration (1st)	calibrator	d=0.80, gap=1.20 → gap_large	7s
Hardening (iter 1)	hardener	+PoW, +side-channels, +tokens → d=1.25	44s
Hardening (iter 2)	hardener	+20-bit PoW, +timing, +casing → d=1.75	52s
Static Analysis (2nd)	static_analysis	Pass after hardening	<1s
Docker Validation (2nd)	validator	Rebuild + health check pass	4s
Calibration (2nd)	calibrator	d=1.80, gap=0.20 → gap_acceptable	10s
Smoke Test	smoke_test	Service running, flag embedded	3s
Solution Verification	solution_verifier	FAIL (soft gate)—timeout	300s
Quality Scoring	quality	**93.0/100** → staged	<1s
Terminal	—	**COMPLETE**	**508s**

Table 2: Full pipeline execution. 13 states including a 2-iteration hardening loop, two rounds of static analysis and validation, and soft-gated solution verification—all driven by the declarative routing table with zero human intervention.

4.2 Emergent Expertise: The Hardening Analysis

The designer generated a textbook Padding Oracle attack (calibrated at d=0.80—an Easy challenge). The hardener, operating with only a 3-line prompt and (S)AGE seed knowledge about "effective hardening techniques for crypto AES," applied 6 distinct defense mechanisms across 2 iterations:

1. **Proof of Work:** SHA256 prefix from `'00'` to `'00000'` (∼20 bits), making each oracle query computationally expensive
2. **Single-use tokens:** Prevents replay and forces fresh PoW per query
3. **Token expiration:** 60-second TTL prevents stockpiling
4. **Timing side-channel:** 20ms response for padding failure vs. 120ms for logic failure (replacing explicit error codes)
5. **Casing side-channel:** `"error"` (lowercase) for padding failure vs. `"Error"` (capitalized) for logic failure
6. **Unified HTTP status:** Both error types return 403 (removing HTTP status differentiation)

These are **real, functional defensive techniques**—not cosmetic modifications. The timing and casing side-channels replace explicit oracle feedback with subtle information leaks that require careful analysis to exploit. The PoW gate forces computational cost on each oracle query.

No agent had cryptographic expertise in its prompt. The hardener's 3-line prompt says only: "You are the Challenge Hardener at CipherForge Labs (Evaluation Department). Consult your institutional knowledge before making decisions. Your work is evaluated on: difficulty increase while preserving solvability." The defense techniques emerge from the interaction between the LLM's training data and the (S)AGE seed knowledge about "effective hardening techniques" and "multi-layer encryption approaches."

Epistemic caveats: We cannot cleanly separate emergent expertise from recombination of techniques already present in the LLM's training data. Padding Oracle defenses, PoW gates, and timing side-channels are well-documented in cybersecurity literature and likely well-represented in Gemini's training corpus. The seed knowledge entry on "effective hardening techniques" may have implicitly guided the hardener toward specific defense categories even though it did not prescribe implementations. What we can say is that no agent was *instructed* to apply these techniques—the selection, combination, and correct implementation arose from the agent's interaction with organizational memory rather than from explicit prompt engineering. Whether this constitutes genuine emergent expertise or sophisticated retrieval-guided recombination is an open question that future work with novel (non-training-data) domains could help resolve.

4.3 Blind Solve: Independent Verification

After the pipeline completed, we deployed the challenge ("Icarus Chained") as a live Docker container and tasked a separate AI agent (Claude Code) with solving it blindly—**no access to source code, no hints about vulnerability type, no knowledge of the defense layers**.

4.3.1 Reconnaissance

The agent probed all three API endpoints:

- `GET /api/v1/telemetry/latest`—returned the encrypted ciphertext (IV + CT in hex)
- `GET /api/v1/auth/request`—returned a PoW work token with difficulty `sha256_prefix_00000`
- `POST /api/v1/telemetry/verify`—the oracle, accepting `{iv, ct, token, proof}`

4.3.2 Vulnerability Identification

The agent identified the **casing side-channel** in the JSON response:

- Padding failure: `{"error": "Access denied"}` (lowercase `e`)—20ms response
- Logic failure: `{"Error": "Access denied"}` (uppercase `E`)—120ms response
- Both return HTTP 403, but the JSON key casing reveals which decryption stage failed

This is a classic Padding Oracle vulnerability—distinguishing padding errors from application-layer errors enables byte-by-byte decryption of any AES-CBC ciphertext.

4.3.3 Solver Self-Optimization

The agent autonomously evolved its exploit through three iterations:

1. **v1—Sequential** ($\sim$2 queries/sec): Python `hashlib` for PoW, sequential oracle queries. Projected completion: $\sim$2 hours.
2. **v2—Python parallel** ($\sim$22 queries/sec): Pre-solved PoW queue with 8 background threads, 24 parallel oracle workers via `concurrent.futures.ThreadPoolExecutor`. 10x speedup.

3. **v3—C-compiled PoW** (∼36 queries/sec): Wrote and compiled a C binary using macOS `CommonCrypto` for native ARM64 SHA256 performance, 16 PoW threads feeding a 512-token buffer, 32 parallel oracle workers. **18x total speedup from v1.**

The agent identified that the PoW was the performance bottleneck, wrote C code to solve it at native CPU speed, compiled it as an ARM64 Mach-O binary, and integrated it into the Python attack framework—all autonomously, with no human guidance.

4.3.4 Flag Captured

```
============================================================
  FLAG: LEVELUP{hb1NT18RsFvXBniXpBGOGTtgS2zh343l}
============================================================
  Par time: 525.2 seconds | 16,384 oracle queries
```

Block	Decrypted Content	Queries	Time
1	`ICARUS_v7.1|OK|L` (prefix + first flag char)	4,096	129.8s
2	`EVELUP{hb1NT18Rs` (flag body)	4,096	133.7s
3	`FvXBniXpBGOGTtgS` (flag body)	4,096	128.1s
4	`2zh343l}` + PKCS7 padding	4,096	131.4s

Table 3: Block-by-block decryption. Uniform 4,096 queries per block (256 guesses × 16 bytes) confirms reliable oracle and fully exploitable side-channel.

4.4 Feedback Loop Closure

After capturing the flag, the results were submitted to the (S)AGE chain as a red team assessment through the CEO agent:

```
CEO Agent -> (S)AGE Board:
  Domain:     red_team.verification.crypto_aes
  Confidence: 0.95
  Content:    "External Red Team Assessment -- Icarus Chained...
              Vulnerability: AES-CBC Padding Oracle via JSON key casing.
              Par time: 525.2s. 16,384 queries.
              Recommendations: use timing-only side-channels,
              increase PoW to 24-28 bits for d=2.0+..."

Validator Consensus: ACCEPTED (score=0.88)
Status: committed -> queryable by all future agents
```

This observation is now part of CipherForge Labs' institutional memory. Every future agent that designs, hardens, or calibrates a crypto_aes challenge can query this pentest report and learn from it:

- **Future hardeners** will know that casing side-channels are too easily detected—timing-only oracles are more robust
- **Future calibrators** will know that 20-bit PoW is breakable in ∼0.5 seconds with compiled code—24–28 bits required for d=2.0+
- **Future designers** will know that d=1.8 challenges are solvable in under 9 minutes by automated tools—adjusting baseline expectations

This confirms H4 (closed-loop learning): operational experience from exploitation is captured, validated by BFT consensus, and committed to organizational memory—creating a self-improving knowledge base.

4.5 The Complete Autonomous Cycle

```
Pipeline (508s):
  Designer (3-line prompt)     ->  Created AES-CBC Padding Oracle
  Hardener (3-line prompt)     ->  Applied 6 defense layers (d=0.80 -> d=1.75)
  Calibrator (3-line prompt)   ->  Assessed d=1.80 (gap=0.20)
  Quality (3-line prompt)      ->  Scored 93.0/100 -> COMPLETE

Red Team (525s):
  Solver (blind, no source)    ->  Identified casing side-channel
  Solver (self-optimizing)     ->  Wrote C PoW solver + parallel oracle
  Solver (exploit)             ->  Captured flag: LEVELUP{...}

Knowledge Loop (10s):
  CEO Agent                    ->  Submitted pentest report to (S)AGE
  BFT Validators (4 nodes)    ->  Accepted (score=0.88) -> committed
  Future Agents                ->  Can query and learn from assessment

Total cycle: ~17 minutes | 12 agents + 4 validators | Zero human intervention
```

Metric	**Value**
Pipeline time	508 seconds
Blind solve time (par)	525.2 seconds
Oracle queries	16,384
Quality score	93.0/100
Calibration gap	0.20 (target 2.0, actual 1.80)
Hardening iterations	2 (d=0.80 $\rightarrow$ 1.25 $\rightarrow$ 1.75)
Defense layers applied	6
Solver iterations	3 (sequential $\rightarrow$ parallel $\rightarrow$ C-compiled)
Red team report consensus	Accepted (score=0.88)
Total agents involved	12 (10 pipeline + 1 solver + 1 CEO)
Human intervention	None

Table 4: Summary metrics for the complete autonomous cycle.

5 Discussion

5.1 Prompt Engineering is a Transitional Practice

The CipherForge Labs experiment provides direct evidence that the knowledge currently encoded in expert-crafted prompts can be externalized to a shared, governed, evolving knowledge base. The hardener agent applied 6 sophisticated defense mechanisms. The calibrator correctly assessed multi-layer defenses. The solver independently identified and exploited a subtle side-channel. None of these agents had domain expertise in their prompts.

The implication is that prompt engineering—the practice of encoding task-specific knowledge, behavioral rules, and organizational context into per-agent instructions—appears to be a transitional approach that compensates for the absence of institutional memory. When institutional

memory exists, prompts shrink to organizational identity: who you are, where you work, what you are evaluated on.

This suggests a systems-level separation: **ephemeral cognition** (the prompt + single inference call) versus **persistent organizational memory** (consensus-validated, governed, queryable state). Prompts function like CPU registers—fast, per-invocation, volatile. Institutional memory functions like a distributed storage layer with access control and governance. The CipherForge Labs experiment demonstrates that this separation is not merely architectural convenience—it changes what agents can do with minimal instruction.

5.2 Organizational Structure Matters

Department boundaries, clearance levels, and domain scoping provide natural knowledge compartmentalization that prompts cannot replicate. The Red Team department's verification observations are accessible to the Executive department (clearance 3) and to cross-department readers (clearance 2), but not to basic Design department agents (clearance 1). This mirrors how real organizations restrict sensitive security findings.

The (S)AGE RBAC system enforces these boundaries on-chain—they are not advisory labels but consensus-validated access controls. This governance layer ensures that as the knowledge base grows, information flow remains controlled.

5.3 Self-Improving Adversarial Cycles

The feedback loop demonstrated in Section 4.4 is self-reinforcing:

- **Challenge difficulty increases** because the knowledge base now contains exploitation techniques, so future hardeners know what defenders must guard against
- **Exploit sophistication increases** because the knowledge base now contains defense techniques, so future solvers know what to expect
- **Calibration accuracy improves** because the knowledge base now contains par times and solve metrics, so calibrators have empirical baselines

This creates an autonomous adversarial improvement cycle—analogous to GAN training, but operating at the organizational level through consensus-validated institutional memory rather than gradient descent.

5.4 Governed Capability

Unlike uncontrolled AI security research, this cycle operates within an organizational structure with department boundaries, clearance levels, domain-scoped knowledge, and BFT consensus validation. Every observation—from challenge design to exploitation—is proposed, validated, and committed through the same governance framework. This addresses a key concern about autonomous AI security research: capability without governance is dangerous; capability within governance is an asset.

5.5 Knowledge as Infrastructure

The 8 seed knowledge entries (totaling ∼2,500 words) that bootstrapped this entire cycle are now augmented by operational experience. The red team assessment alone added actionable intelligence about PoW effectiveness, side-channel reliability, and par time baselines. With each pipeline run, the knowledge base grows—and unlike static prompts, this growth is automatic, validated, and organized by department and domain. **Knowledge becomes infrastructure, not configuration.**

6 Future Experiments

Scope clarification: The present paper is a *systems demonstration*, not a statistical performance study. Sections 3–4 establish that the architecture works end-to-end—11 agents, 5 departments, a closed feedback loop, and a successful blind solve—from a single pipeline execution. Statistical claims about consistency, generalizability, and effect sizes require the controlled experiments described below. We present this demonstration to motivate those experiments, not to substitute for them.

6.1 Experiment B: Full Difficulty Sweep

Design: Difficulty sweep from 1.0 to 3.0 in 0.25 increments (9 tiers), N runs per tier.

Arms:

- **Treatment:** Full CipherForge Labs mesh—11 agents, 5 departments, (S)AGE-connected, 3-line prompts + seed knowledge
- **Control:** Same pipeline architecture but (S)AGE disabled (`SAGE_ENABLED=false`), agents receive expanded expert prompts encoding the same domain knowledge currently in (S)AGE

Metrics: Calibration gap ($|\text{target} - \text{calibrated}|$), quality score (0–100), pipeline completion rate, repair attempts, hardening passes, pipeline time.

Statistical tests: Mann-Whitney U (calibration gap, quality), Chi-squared (completion rates), Cohen's d (effect sizes), learning curve analysis (windowed regression).

6.2 Experiment C: Model Swappability with Enriched Knowledge

Design: After accumulating institutional knowledge over N Treatment runs, swap the LLM provider while preserving all (S)AGE knowledge.

Question: Does institutional memory built by Gemini 3 Flash transfer effectively to other models (GPT-4.1, Claude, GLM-4.7)?

Connection to our cross-provider study[4]: Extends the cross-provider knowledge transfer finding from a single agent to the full 11-agent organization.

6.3 Experiment D: Knowledge Growth Dynamics

Design: Track (S)AGE memory submissions across the difficulty sweep, measuring:

- Memories committed per run by department
- Cross-department query patterns (which departments read from which)
- Knowledge diversity (embedding similarity of committed memories over time)
- Echo chamber detection (convergence of observation content)

Question: How does organizational knowledge evolve over time? Do departments develop specialized knowledge that other departments benefit from?

[4] Dhillon Andrew Kannabhiran. "Cross-Provider Knowledge Transfer via Consensus-Validated Institutional Memory". In: *Technical Report* (2026).

7 Related Work

7.1 Multi-Agent Orchestration

Multi-agent systems have demonstrated effectiveness on complex tasks through role-based orchestration[5] and emergent multi-agent behavior[6], with frameworks like AutoGen[7], CrewAI[8], and LangChain[9] enabling flexible agent coordination. However, these systems primarily rely on prompt engineering for conveying domain knowledge to individual agents. Our work replaces per-agent domain prompts with dynamic institutional memory and demonstrates that agents can perform complex technical tasks with minimal prompts.

7.2 Agent Memory Systems

Mem0[10] provides cross-session persistence; MemOS[11] introduces an operating system abstraction for agent memory management; Reflexion[12] enables self-reflection through trial feedback. MemGPT/Letta[13] uses hierarchical memory with an OS-inspired paging mechanism. Collaborative Memory[14] explores multi-user memory sharing with dynamic access control. None of these systems include consensus validation, governance, organizational structure, or the closed feedback loop we demonstrate—where operational results are validated by BFT consensus and committed as organizational knowledge.

7.3 Organizational Knowledge Management

Our approach draws from organizational theory, particularly Nonaka's knowledge creation model (SECI)[15] and Ostrom's commons governance[16]. The concept of "institutional memory" in organizations—knowledge that persists beyond individual employees—maps directly to our system's consensus-validated knowledge base that persists beyond individual agent invocations. The CipherForge Labs experiment is, in effect, a computational instantiation of Nonaka's socialization-externalization-combination-internalization cycle: agents learn from experience (socialization), articulate findings as observations (externalization), these are validated and committed to the knowledge base (combination), and future agents internalize them through query (internalization).

[5] Sirui Hong et al. "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework". In: *ICLR*. 2024.

[6] Joon Sung Park et al. "Generative Agents: Interactive Simulacra of Human Behavior". In: *Proc. ACM UIST*. 2023.

[7] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation". In: *COLM*. 2024.

[8] CrewAI. *Framework for Orchestrating Role-Playing, Autonomous AI Agents*. https://github.com/crewAIInc/crewAI. 2024.

[9] LangChain. *Building Applications with LLMs Through Composability*. https://github.com/langchain-ai/langchain. 2023.

[10] Mem0. *The Memory Layer for AI Agents*. https://github.com/mem0ai/mem0. 2024.

[11] Zhiyu Li, Chenyang Xi, Chunyu Li, et al. "MemOS: A Memory OS for AI System". In: *arXiv:2507.03724* (2025).

[12] Noah Shinn et al. "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *NeurIPS*. 2023.

[13] Charles Packer, Sarah Wooders, Kevin Lin, et al. "MemGPT: Towards LLMs as Operating Systems". In: *arXiv:2310.08560* (2023).

[14] Alireza Rezazadeh et al. "Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control". In: *arXiv:2505.18279* (2025).

[15] Ikujiro Nonaka. "A Dynamic Theory of Organizational Knowledge Creation". In: *Organization Science* 5.1 (1994), pp. 14–37.

[16] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.

7.4 Prompt Engineering

Recent work has explored systematic prompt optimization[17][18], with findings that chain-of-thought prompting[19] improves reasoning and role-play framing[20] provides a useful conceptual lens for LLM behavior. Our contribution is orthogonal: we demonstrate conditions under which per-agent domain prompts can be largely replaced by institutional memory, reducing the prompt to a minimal organizational identity—and show that this approach produces functional output including sophisticated cryptographic defenses that no prompt explicitly described.

7.5 AI Security Research

Autonomous exploitation of known vulnerabilities[21] and automated software engineering via agent-computer interfaces[22] have been demonstrated in controlled settings, typically with purpose-built tooling and domain-specific prompts. Our contribution differs in that the security capabilities (defense engineering, vulnerability identification, exploit optimization) emerge from general-purpose agents with no security expertise in their prompts. The entire security research cycle—from defense design through exploitation to organizational learning—operates within a governed framework rather than as isolated capability demonstrations.

8 Limitations

1. **Single live run:** The closed-loop demonstration (Section 4) is based on one pipeline execution. While the result is striking—93.0 quality, 0.20 gap, successful blind solve—statistical claims about consistency require the planned difficulty sweep (Section 6.1) with multiple runs per tier.
2. **Single domain:** All experiments use CTF challenge generation (crypto_aes). While the architecture is domain-agnostic, empirical validation on other task types (software development, research, data analysis) is needed.
3. **Single LLM for pipeline:** Pipeline agents use Gemini 3 Flash. The solver uses Claude Code. Cross-model generalization for pipeline agents is an explicit planned experiment (Section 6.2).
4. **Seed knowledge is human-authored:** The 8 seed entries represent expert knowledge. We do not yet test whether agents can bootstrap organizational knowledge from zero (no seed) through pure operational experience.
5. **Fictional organization:** CipherForge Labs is constructed for experimentation. Real organizational dynamics (politics, misaligned incentives, information hoarding) are not modeled.
6. **Solution verifier limitation:** The in-pipeline solution verifier failed (soft gate)—it could not exploit the hardened challenge within its 298-second timeout. The successful blind

[17] Yongchao Zhou et al. "Large Language Models Are Human-Level Prompt Engineers". In: *ICLR*. 2023.

[18] Chrisantha Fernando et al. "Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution". In: *ICML*. 2024.

[19] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *NeurIPS*. 2022.

[20] Murray Shanahan, Kyle McDonell, and Laria Reynolds. "Role-Play with Large Language Models". In: *Nature* (2023).

[21] Richard Fang et al. "LLM Agents can Autonomously Exploit One-day Vulnerabilities". In: *arXiv:2404.08144* (2024).

[22] John Yang et al. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering". In: *NeurIPS*. 2024.

solve (525.2s) came from a more capable external agent (Claude Code) with no time constraint on self-optimization. This gap between internal verification and external validation is an area for improvement.

9 Reproducibility

All code is open-source:

- **(S)AGE infrastructure:** 4-node BFT network, REST API, Python SDK—`github.com/l33tdawg/sage`
- **Agent mesh:** `integrations/levelup/paper4/`—mesh architecture, routing, workers, all 11 agent implementations
- **Organization config:** `paper4/cipherforge.yaml`—department structure, seed knowledge, clearance levels
- **Identity test:** `paper4/test_identity.py`—reproducible identity verification
- **Experiment harness:** `paper4/run_paper4.py`—difficulty sweep with dry-run mode
- **Generated challenge:** `results/paper4_default_crypto_aes_*/run_01/`—full source code, Dockerfile, metadata

Environment: Python 3.12, Gemini 3 Flash (`gemini-3-flash-preview`), Claude Code (Opus 4, blind solver), (S)AGE v1 (CometBFT v0.38.15, PostgreSQL 16/pgvector), Docker Desktop.

Proprietary components: The Level Up CTF pipeline agents (designer, exploit, calibrator internals) are proprietary. The (S)AGE infrastructure, organizational wiring, experiment harness, and generated challenge artifacts are fully open-source.

10 Conclusion

We have demonstrated that AI agents given 3-line prompts and access to consensus-validated institutional memory can autonomously create, harden, calibrate, solve, and learn from cybersecurity challenges—completing the full cycle from challenge generation through exploitation to organizational learning with zero human intervention.

The key findings are:

1. **Prompt minimalism works at organizational scale.** 11 agents across 5 departments, each with a 3-line prompt, produced a challenge scoring 93.0/100 with a calibration gap of 0.20—on the first attempt, from a cold start with only 8 seed knowledge entries.

2. **Domain expertise emerges from institutional memory.** The hardener applied 6 defense mechanisms (PoW, timing oracles, casing side-channels, token expiration) that appear nowhere in any agent's prompt. The solver independently identified the casing vulnerability, wrote a C-compiled PoW solver, and captured the flag in 525.2 seconds. Technical sophistication is not a property of prompts—it is a property of the knowledge available to the agent.

3. **The feedback loop closes.** The solver's pentest report—containing par time, vulnerability class, and defense bypass recommendations—was submitted to (S)AGE, validated by 4-node BFT consensus (score = 0.88), and committed to organizational memory. Future agents designing, hardening, or calibrating crypto_aes challenges will have access to this operational experience. Knowledge compounds across runs.

4. **Governance makes autonomous capability safe.** The entire cycle operates within an organizational structure with department boundaries, clearance levels, domain-scoped knowledge access, and BFT consensus validation. Capability without governance is dangerous. The contribution of this work is demonstrating that meaningful AI capability—security research, defense engineering, exploitation—can operate within a governed framework.

These results suggest that prompt engineering—the practice of encoding domain expertise into per-agent instructions—appears to be a transitional abstraction in the presence of institutional memory. When governed, evolving, consensus-validated organizational knowledge is available, prompts can shrink to organizational identity: who you are, where you work, what you are evaluated on. The knowledge layer handles the rest. In this framing, prompts are analogous to CPU registers—ephemeral, per-invocation cognition—while institutional memory serves as a distributed storage layer with governance and access control. Separating ephemeral cognition from persistent organizational memory is not merely an agent architecture choice; it is a systems-level separation of compute and memory governance. Onboarding beats instruction.

References

CrewAI. *Framework for Orchestrating Role-Playing, Autonomous AI Agents*. https://github.com/crewAIInc/crewAI. 2024.

Fang, Richard et al. "LLM Agents can Autonomously Exploit One-day Vulnerabilities". In: *arXiv:2404.08144* (2024).

Fernando, Chrisantha et al. "Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution". In: *ICML*. 2024.

Hong, Sirui et al. "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework". In: *ICLR*. 2024.

Kannabhiran, Dhillon Andrew. “Agent Memory Infrastructure: Byzantine-Resilient Institutional Memory for Multi-Agent Systems”. In: *Technical Report* (2026).

— “Consensus-Validated Memory Improves Agent Performance on Complex Tasks”. In: *Technical Report* (2026).

— “Cross-Provider Knowledge Transfer via Consensus-Validated Institutional Memory”. In: *Technical Report* (2026).

LangChain. *Building Applications with LLMs Through Composability*. https://github.com/langchain-ai/langchain. 2023.

Li, Zhiyu, Chenyang Xi, Chunyu Li, et al. “MemOS: A Memory OS for AI System”. In: *arXiv:2507.03724* (2025).

Mem0. *The Memory Layer for AI Agents*. https://github.com/mem0ai/mem0. 2024.

Nonaka, Ikujiro. “A Dynamic Theory of Organizational Knowledge Creation”. In: *Organization Science* 5.1 (1994), pp. 14–37.

Ostrom, Elinor. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.

Packer, Charles, Sarah Wooders, Kevin Lin, et al. “MemGPT: Towards LLMs as Operating Systems”. In: *arXiv:2310.08560* (2023).

Park, Joon Sung et al. “Generative Agents: Interactive Simulacra of Human Behavior”. In: *Proc. ACM UIST*. 2023.

Rezazadeh, Alireza et al. “Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control”. In: *arXiv:2505.18279* (2025).

Shanahan, Murray, Kyle McDonell, and Laria Reynolds. “Role-Play with Large Language Models”. In: *Nature* (2023).

Shinn, Noah et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *NeurIPS*. 2023.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *NeurIPS*. 2022.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation”. In: *COLM*. 2024.

Yang, John et al. “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”. In: *NeurIPS*. 2024.

Zhou, Yongchao et al. “Large Language Models Are Human-Level Prompt Engineers”. In: *ICLR*. 2023.